Visual Programming

C# (C sharp)

Expressions & Operators
Control Structures

Lecturer: Dr. Khaled Mohammed BinAbdl

Department of Information Technology,

College of Computers, Seiyun University

Expressions & Operators

- **C# operators:** C# contains a number of operators:
  - The basic mathematical operations, such as the + operator to add two operands.
  - Logical operators specifically for dealing with Boolean values, and
  - Assignment operators like =.
- Three categories of Operators:
  - Unary operators, which act on single operands
  - Binary operators, which act on two operands
  - Ternary operators, which act on three operands
- **Expressions:** The basic building blocks of computation.
  - Created by combining operators with variables and literal values (together referred to as operands when used with operators)

Mathematical Operators

C# operation	Arithmetic operator	Algebraic expression	C# expression
Addition	+	$f + 7$	`f + 7`
Subtraction	-	$p - c$	`p - c`
Multiplication	*	$b \cdot m$	`b * m`
Division	/	x / y or $\frac{x}{y}$ or $x \div y$	`x / y`
Remainder	%	$r \bmod s$	`r % s`

Fig. 3.18 | Arithmetic operators.

Operators	Operations	Order of evaluation (associativity)
Evaluated first		
*	Multiplication	If there are several operators of this type, they're evaluated from left to right.
/	Division	
%	Remainder	
Evaluated next		
+	Addition	If there are several operators of this type, they're evaluated from left to right.
-	Subtraction	

Fig. 3.19 | Precedence of arithmetic operators.

Step 1.	`y = 2 * 5 * 5 + 3 * 5 + 7;` `2 * 5 is 10`	*(Leftmost multiplication)*
Step 2.	`y = 10 * 5 + 3 * 5 + 7;` `10 * 5 is 50`	*(Leftmost multiplication)*
Step 3.	`y = 50 + 3 * 5 + 7;` `3 * 5 is 15`	*(Multiplication before addition)*
Step 4.	`y = 50 + 15 + 7;` `50 + 15 is 65`	*(Leftmost addition)*
Step 5.	`y = 65 + 7;` `65 + 7 is 72`	*(Last addition)*
Step 6.	`y = 72`	*(Low-precedence assignment—place* 72 *in* y*)*

Fig. 3.20 | Order in which a second-degree polynomial is evaluated.

Assignment operator	Sample expression	Explanation	Assigns
Assume: `int c = 3, d = 5, e = 4, f = 6, g = 12;`			
`+=`	`c += 7`	`c = c + 7`	10 to c
`-=`	`d -= 4`	`d = d - 4`	1 to d
`*=`	`e *= 5`	`e = e * 5`	20 to e
`/=`	`f /= 3`	`f = f / 3`	2 to f
`%=`	`g %= 9`	`g = g % 9`	3 to g

Fig. 5.14 | Arithmetic compound assignment operators.

Operator	Sample expression	Explanation
++ (prefix increment)	`++a`	Increments `a` by 1 and uses the new value of `a` in the expression in which `a` resides.
++ (postfix increment)	`a++`	Increments `a` by 1, but uses the original value of `a` in the expression in which `a` resides.
-- (prefix decrement)	`--b`	Decrements `b` by 1 and uses the new value of `b` in the expression in which `b` resides.
-- (postfix decrement)	`b--`	Decrements `b` by 1 but uses the original value of `b` in the expression in which `b` resides.

Fig. 5.15 | Increment and decrement operators.

Manipulating Variables with Mathematical Operators

1. Create a new console application
2. Add the following code to Class1.cs :

```
static void Main(string[] args)
    {
        double firstNumber, secondNumber;
        string userName;
        Console.WriteLine("Enter your name:");
        userName = Console.ReadLine();
        Console.WriteLine("Welcome {0}!", userName);
        Console.WriteLine("Now give me a number:");
        firstNumber = Convert.ToDouble(Console.ReadLine());
        Console.WriteLine("Now give me another number:");
        secondNumber = Convert.ToDouble(Console.ReadLine());
        Console.WriteLine("The sum of {0} and {1} is {2}.", firstNumber,
                secondNumber, firstNumber + secondNumber);
        Console.WriteLine("The result of subtracting {0} from {1} is {2}.",
                secondNumber, firstNumber, firstNumber - secondNumber);
        Console.WriteLine("The product of {0} and {1} is {2}.", firstNumber,
                secondNumber, firstNumber * secondNumber);
        Console.WriteLine("The result of dividing {0} by {1} is {2}.",
                firstNumber, secondNumber, firstNumber / secondNumber);
        Console.WriteLine("The remainder after dividing {0} by {1} is {2}.",
                firstNumber, secondNumber, firstNumber % secondNumber);
    }
```

Namespaces

- They are the .NET way of providing containers for application code, such that code and its contents may be uniquely identified.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace DemoApplication
  {
     class Program
       {
          static void Main(string[] args)
            {
               String message="Hello";
               Console.Write(message);
               Console.ReadKey();
            }
       }
  }
```

namespace

Control Structures

- Two methods of controlling program flow, that is, the order of execution of lines of C# code.
  - Branching – where we execute code conditionally, depending on the outcome of an evaluation, such as "only execute this code if myVal is less than 10".
  - Looping – repeatedly executing the same statements (for a certain number of times or until a test condition has been reached).
- The use of Boolean logic is involved:
  - boolean logic is used in flow control scenarios.
  - Boolean comparisons require the use of Boolean comparison operators (also known as relational
  - operators), see Table on next slide:

Relational Operators:

In all cases here var1 is a bool type variable, while the types of var2 and var3 may vary.

Operator	Category	Example Expression	Result
==	binary	var1 = var2 == var3;	var1 is assigned the value true if var2 is equal to var3, or false otherwise.
!=	binary	var1 = var2 != var3;	var1 is assigned the value true if var2 is not equal to var3, or false otherwise.
<	binary	var1 = var2 < var3;	var1 is assigned the value true if var2 is less than var3, or false otherwise.
>	binary	var1 = var2 > var3;	var1 is assigned the value true if var2 is greater than var3, or false otherwise.
<=	binary	var1 = var2 <= var3;	var1 is assigned the value true if var2 is less than or equal to var3, or false otherwise.
>=	binary	var1 = var2 >= var3;	var1 is assigned the value true if var2 is greater than or equal to var3, or false otherwise.

There are some other Boolean operators that are intended specifically for working with Boolean values:

Operator	Category	Example Expression	Result
!	unary	`var1 = ! var2;`	`var1` is assigned the value `true` if `var2` is `false`, or `false` if `var2` is `true`. (Logical NOT.)
&	binary	`var1 = var2 & var3;`	`var1` is assigned the value `true` if `var2` and `var3` are both `true`, or `false` otherwise. (Logical AND.)
|	binary	`var1 = var2 | var3;`	`var1` is assigned the value `true` if either `var2` or `var3` (or both) are `true`, or `false` otherwise. (Logical OR.)
^	binary	`var1 = var2 ^ var3;`	`var1` is assigned the value `true` if either `var2` or `var3`, but not both, are `true`, or `false` otherwise. (Logical XOR, or exclusive OR.)

Operator	Category	Example Expression	Result
&&	binary	`var1 = var2 && var3;`	`var1` is assigned the value `true` if `var2` and `var3` are both `true`, or `false` otherwise. (Logical AND.)
||	binary	`var1 = var2 || var3;`	`var1` is assigned the value `true` if either `var2` or `var3` (or both) are `true`, or `false` otherwise. (Logical OR.)

Branching – if Statement

- Branching is the act of controlling which line of code should be executed next according to some kind of conditional statement - a comparison between a test value and one or more possible values using Boolean logic.

```
if (<test>)
   <code executed if <test> is
true>;
else
   <code executed if <test> is
false>;
```

```
if (<test>)
{
   <code executed if <test> is true>;
}
else
{
   <code executed if <test> is false>;
}
```

1. Create a new console application.
2. Add the following code to Class1.cs:

```
static void Main(string[] args)
    {
      string comparison;
      Console.WriteLine("Enter a number:");
      double var1 = Convert.ToDouble(Console.ReadLine());
      Console.WriteLine("Enter another number:");
      double var2 = Convert.ToDouble(Console.ReadLine());
      if (var1 < var2)
        comparison = "less than";
      else
      {
        if (var1 == var2)
          comparison = "equal to";
        else
           comparison = "greater than";
      }
     Console.WriteLine("The first number is {0} the second number.", comparison);
    }
```

Branching –switch Statement

```
switch (<testVar>)
{
   case <comparisonVal1>:
      <code to execute if <testVar> == <comparisonVal1> >
      break;
   case <comparisonVal2>:
      <code to execute if <testVar> == <comparisonVal2> >
      break;
   ...
   case <comparisonValN>:
      <code to execute if <testVar> == <comparisonValN> >
      break;
   default:
      <code to execute if <testVar> != comparisonVals>
      break;
}
```

1. Create a new console application.
2. Add the following code to Class1.cs:

```
static void Main(string[] args)
    {
      const string myName = “ali";
      const string funnyName = “zozo";
      const string sillyName = “zebro";
      string name;
      Console.WriteLine("What is your name?");
      name = Console.ReadLine();
      switch (name.ToLower ())
      {
        case myName:
          Console.WriteLine("You have the same name as me!");
          break;
        case funyName:
          Console.WriteLine("My, what a funny name you have!");
          break;
        case sillyName:
          Console.WriteLine("That's a very silly name.");
          break;
      }
      Console.WriteLine("Hello {0}!", name);
    }
```

Looping

- Looping is where statements are executed repeatedly.
- **Do..while Loops**

  The code, marked out for looping, is executed, then a Boolean test is performed, and the code executes again if this test evaluates to true, otherwise the loop exits.
- The structure of a do loop:

```
do
  {
    <code to be looped>
  } while (<Test>);
```

For example

```
int i = 1;
do
{
  Console.WriteLine("{0}", i++);
} while (i <= 10);
```

Where <Test> evaluates to a Boolean value.

1. Create a new console application.
2. Add the following code to Class1.cs:

```
static void Main(string[] args)
  {
    double balance, interestRate, targetBalance;
    Console.WriteLine("What is your current balance?");
    balance = Convert.ToDouble(Console.ReadLine());
    Console.WriteLine("What is your current annual interest rate (in %)?");
    interestRate = 1 + Convert.ToDouble(Console.ReadLine()) / 100.0;
    Console.WriteLine("What balance would you like to have?");
    targetBalance = Convert.ToDouble(Console.ReadLine());
    int totalYears = 0;
    do
    {
      balance *= interestRate;
      ++totalYears;
    }
    while (balance < targetBalance);
    Console.WriteLine("In {0} year{1} you'll have a balance of {2}.",
                      totalYears, totalYears == 1 ? "" : "s", balance);
  }
```

while Loops

- Very similar to do..while loop, but have one important difference. The Boolean test in a while loop takes place at the start of the loop cycle, not the end. If the test evaluates to false then the loop cycle is never executed.

- while loops are specified in the following way:

```
while (<Test>){
   <code to be looped>
}
```

For example

```
int i = 1;
while (i <= 10)
{
   Console.WriteLine("{0}", i++);
}
```

Exercise: change the c# program on the previous slide to use while loop in place of do..while loop.

For Loops

- A type of loop that executes a set number of times, and maintains its own counter.
- To define a for loop we need the following information:
  - A starting value to initialize the counter variable (<initialization>
  - A condition for continuing the loop, involving the counter variable
  - An operation to perform on the counter variable at the end of each loop cycle
  - for (<initialization>; <condition>; <operation>)

```
{
    <code to loop>
}
```

For example

```
int i;
for (i = 1; i <= 10; ++i)
{
   Console.WriteLine("{0}", i);
}
```

1. Create a new console application.
2. Add the following code to Program .cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace DemoApplication
 {
    class Program
     {
        static void Main(string[] args)
          {
             for(Int32 i=0;i<3;i++)
              {
                   Console.WriteLine(i);
              }
              Console.ReadKey();

          }
    }
 }
```